Structures de données et algorithmes fondamentaux

Contrôle court

7 décembre 2022

Durée 2h. AUCUN DOCUMENT N'EST AUTORISÉ. Il sera tenu compte de la rédaction. Le barème est donné à titre indicatif et est susceptible de changer. Les exercices sont indépendants. *Il est possible de ne répondre qu'à certaines questions d'un exercice. Il est aussi possible de donner la complexité d'un algorithme (ou fonction) que l'on n'a pas défini.*

Rappel : L'instruction `randint(a,b)` retourne un entier aléatoire de l'intervalle $[a, b]$.

Exercice 1 : Variance (9 points)

Étant donné un ensemble de valeurs $X = x_1, \ldots, x_n$, la variance permet de déterminer si les valeurs sont très dispersées (variance importante) ou non. Elle se calcule de la manière suivante :

$$\sum_{j=1}^{n} (E(X) - x_j)^2$$

où $E(X)$ est la moyenne des valeurs $x_1, \ldots, x_n$.

On souhaite calculer la variance d'un ensemble de valeurs. Cet ensemble sera représenté sous la forme d'un tableau de nombres flottants.

Question 1.1 : (2 points) Définir la fonction `moyenne` prenant en paramètre un tableau de nombres et retournant la moyenne des nombres de ce tableau.

Question 1.2 : (1 point) Quelle est la complexité asymptotique de la fonction moyenne ? Justifiez.

Question 1.3 : (1 point) Voici le code de la fonction `variance` :

```
def variance(tab):
  var = 0
  j = 0
  while j < len(tab):
    var += (moyenne(tab) - tab[j]) ** 2
    j += 1
  return var
```

Remarque : `x ** 2` correspond à x^2 en python.

Quelle est la complexité asymptotique de cet algorithme ? Justifiez.

Question 1.4 : (2 points) Peut-on améliorer cette complexité ? Proposer une nouvelle fonction `variance2` calculant la variance de manière plus efficace.
Quelle est la complexité asymptotique de la fonction `variance2` ? Justifiez.

Question 1.5 : (3 points) On suppose que les trois fonctions sont définies dans le fichier `statistique.py` situé dans le répertoire `Analyse`. Dans le même répertoire que celui

contenant le répertoire `Analyse` se trouve le fichier `exercice1.py`. Écrire le code du fichier `exercice1.py`. Ce script doit créer un tableau de 10000 nombres entiers aléatoires entre 0 et 100 inclus. Il doit ensuite mesurer et afficher les temps d'exécution en millisecondes nécessaires pour calculer l'écart-type des nombres du tableau avec la fonction `variance` puis `variance2`. Le répertoire `Analyse` et le fichier `statistique.py` joueront respectivement le rôle de package et module.

Exercice 2 : Jeu de la vie (11 points)

Le jeu de la vie n'est pas réellement un jeu. Il n'y a en effet aucun joueur. C'est un modèle qui simule l'évolution de la vie et la mort de cellules. Le jeu se déroule sur une grille à deux dimensions. Chaque case, appelée *cellule* par analogie avec les cellules vivantes, peut prendre deux états distincts : **vivante** ou **morte**. Au départ (étape 0), l'état de chaque cellule sur la grille est donné aléatoirement. À chaque étape, l'évolution d'une cellule est entièrement déterminée par l'état de ses huit cellules voisines à l'étape précédente de la façon suivante :

— Une cellule morte possédant exactement trois voisines vivantes devient vivante (elle naît).

— Une cellule vivante possédant deux ou trois voisines vivantes le reste, sinon elle meurt.

Le jeu de la vie consiste à déterminer quel est l'état des cellules au bout d'un certain nombre d'étapes.

La figure 1 montre un exemple du jeu de la vie. À l'étape 0, quatre cellules sont vivantes (représentées en noir). À l'étape 1, la cellule de la ligne 3 et de la colonne 4 devient vivante puisqu'à l'étape 0, cette cellule a exactement 3 voisines vivantes (les 3 de la colonne 3). Par contre, la cellule de la ligne 5 et de la colonne 4 meurt à l'étape 1 puisqu'à l'étape précédente, elle n'avait qu'un seul voisin.

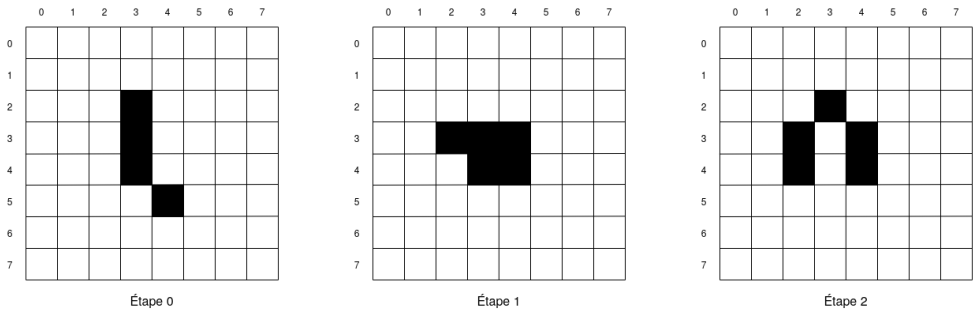

FIGURE 1 – Exemple du jeu de la vie sur une grille 8×8 pour les étapes 0, 1 et 2. Les cellules en noir sont vivantes.

Le but de cet exercice est de programmer le jeu de la vie. La grille utilisée sera un tableau d'entiers à deux dimensions de taille $\mathtt{n} \times \mathtt{n}$. Chaque case du tableau sera égale à 1 si la cellule est vivante, et 0 si elle est morte.

Question 2.1 : (3 points) Définir la fonction `initialise` qui prend en paramètre un entier `n` et retourne une grille $n \times n$ où les cases sont initialisées à 0 ou 1 de manière aléatoire.

Question 2.2 : (2.5 points) Définir la fonction `affiche` prenant en paramètre une grille (correspondant à une étape du jeu de la vie) et l'affichant. Le symbole "X" sera utilisé pour indiquer que la cellule est vivante et le symbole "_" que la cellule est morte. Si la grille est celle de l'étape 0, la fonction doit afficher :

```
_ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _
_ _ _ X _ _ _ _
_ _ _ X _ _ _ _
_ _ _ X _ _ _ _
_ _ _ _ X _ _ _
_ _ _ _ _ _ _ _
_ _ _ _ _ _ _ _
```

Question 2.3 : (2.5 points) Définir la fonction `sera_vivante` prenant en paramètre une grille ainsi que les indices de ligne et colonne d'une cellule, et retournant **`True`** si la cellule sera vivante à l'étape suivante et **`False`** sinon. L'appel de la fonction `sera_vivante` avec la grille correspondant à l'étape 0 et les indices 3 (ligne) et 4 (colonne) doit retourner **`True`**.

Question 2.4 : (2 points) Définir la fonction `nouvelle_etape` prenant en paramètre une grille. La fonction calcule l'étape suivante à partir de cette grille et la modifie en conséquence.

Rappel : La fonction `deepcopy(sd)` retourne une copie de la structure de données imbriquée `sd` passée en paramètre.

Question 2.5 : (1 point) Écrire le code initialisant une grille 10×10 et affichant l'état de la grille après 200 étapes.